

MIOS Mode Monitor — PoC Run Guide

Project: Market Intelligence Operating System (MIOS) — Mode Monitor Proof of Concept, Easy Skill Australia.

Audience: Member 5 (build owner). Step-by-step instructions for running the PoC end-to-end on your local machine, plus a reference of what every code file does.

Repository root: E:\UniProjects\MIOS

Python: 3.11 or later. Tested on 3.14 / Windows 11.

Note. AI assistance disclosure. The PoC scaffolding was built with assistance from Claude Code (Anthropic). All architectural decisions, prompt design, dataset curation, and final review were authored by the project team.

1. How to run the PoC

1.1 First-time setup

Open a terminal at the repository root.

```
cd /e/UniProjects/MIOS
python -m pip install -e .[dev]
```

This installs the runtime dependencies (google-genai, crawlee[beautifulsoup], rapidfuzz, requests, python-dotenv) plus pytest for the dev extras.

Configure secrets

Copy the env template, then edit .env with your real values. .env is gitignored and must never be committed.

```
cp .env.example .env
```

GEMINI_API_KEY	Get from https://aistudio.google.com (free tier, no card needed).
GEMINI_MODEL	Default gemini-2.5-flash. Switch to gemini-2.5-flash-lite if the daily quota of the main model is exhausted — it has a separate quota pool.
SLACK_WEBHOOK_URL	Create at https://api.slack.com/apps → Create App → Incoming Webhooks → Activate → Add New Webhook to Workspace → copy the URL.
DB_PATH	Default data/mios.db. The file is created on first run and gitignored.
LOG_LEVEL	Default INFO. Set DEBUG if you want fuzzy-match score traces.
PNGWORKFORCE_BASE_URL	Default https://www.pngworkforce.com .

1.2 Run the unit tests (offline, no API or Slack calls)

```
python -m pytest -q
```

Expect **40 passed**. Tests mock the Gemini client and use a saved HTML fixture for the scraper, so they run with no network.

1.3 End-to-end live run

The KPI harness orchestrates the full pipeline: load synthetic ground truth into SQLite → classify with Gemini → score against ground truth → write `results.csv` → build the weekly digest → post to Slack.

Single run (recommended first)

```
python -m evaluation.kpi_harness --runs 1
```

Runtime: about **9–10 minutes**. The 7 s inter-call delay between Gemini requests is required to stay below the 10 RPM free-tier ceiling. Outputs:

- `results.csv` — per-run scores and an aggregate row.
- A markdown KPI table on stdout, matching the §5.2 report format.
- A weekly digest delivered to your Slack channel.

Five runs (for the §5.2 numbers in the report)

```
python -m evaluation.kpi_harness --runs 5
```

Runtime: about 50 minutes. The harness wipes the signals table between runs but keeps the watchlist and writes mean + stdev for the latency KPI.

Useful flags

```
python -m evaluation.kpi_harness --runs 1 --no-slack      # skip Slack delivery
python -m evaluation.kpi_harness --db data/local.db      # custom db path
python -m evaluation.kpi_harness --ground-truth data/my.jsonl
```

If quota is exhausted

The free tier is approximately 200 requests-per-day per model. One full run (80 records minus blocklist filters) uses ~60 calls. If you hit `RESOURCE_EXHAUSTED` 429 errors, switch model or wait until the next day:

```
# Linux / macOS / git-bash
GEMINI_MODEL=gemini-2.5-flash-lite python -m evaluation.kpi_harness --runs 1

# PowerShell
$env:GEMINI_MODEL="gemini-2.5-flash-lite"; python -m evaluation.kpi_harness --runs 1
```

1.4 Standalone live scrape (best-effort)

To attempt a live PNGworkforce fetch:

```
python -c "from scraper.pngworkforce import scrape; r = scrape(limit=10);
print(len(r), 'jobs found')"
```

Returns an empty list and logs a warning if the site's HTML does not match the heuristic selectors — that's the brief's required behaviour. The pipeline still runs against the synthetic dataset regardless.

1.5 After the run — deliverables for the report

- Open `results.csv` and copy the aggregate row into §5.2 of the project report. Replace the three [TBC] phrases:

- "roughly a third" → the actual filtered percentage from the harness logs.
- "noticeably easier to classify" → the real accuracy delta when comparing synthetic-only vs real-data-included runs.

— "unpredictable lag" → the actual `stdev_latency_seconds` across your 5 runs.

- Record a 2–3 minute demo video of the full pipeline running. Save into `demo/`.
- Take screenshots of: SQLite db open in DB Browser, Gemini API call in the console logs, Slack message in your workspace, and `results.csv` in a spreadsheet. Use these in slides 7–9.
- Push the repo to GitHub and link from §5.2 of the report.

2. How each code file works

The repository follows the four-stage pipeline: scrape → load → classify → deliver. Each stage lives in its own package. The KPI harness is the orchestrator; everything else is library code.

config/

config/settings.py

Loads `.env` via `python-dotenv` and exposes a frozen `Settings` dataclass: API keys, model name, db path, log level, scraper base URL, and watchlist path. A module-level settings singleton is imported by every other module — no module re-reads `.env`. The helper `configure_logging()` wires Python's `stdlib` logger to `LOG_LEVEL`.

config/watchlist.json

The 20-company priority list: 10 Tier A active clients, 7 Tier B prospects, 3 Tier C indicators — drawn from the Easy Skill brief (BHP, Rio Tinto, Newmont/Lihir, Barrick/Porgera, TotalEnergies, ExxonMobil, Glencore, Downer, Monadelphous, Ok Tedi at Tier A; Vale, Eramet, Saipem, Technip, Subsea7, Aurecon, Vinci at Tier B; Thales, Safran, Eiffage at Tier C). Each entry carries a sector, notes, and an `aliases` array used by the fuzzy matcher.

loader/

loader/schema.sql

DDL for two tables. The `signals` table mirrors the production BigQuery schema: `signal_id` (UUID), source metadata, geography, sector, watchlist tier, signal category, review cycle, raw content, classification notes, `is_new_prospect` flag, and `classified_at` timestamp. A unique partial index on `source_url` enforces dedupe at the database layer. The `watchlist` table seeds from the JSON file at init time.

loader/ingest.py

Two public functions:

- `init_db(db_path, watchlist_path=None)` — runs the schema script and seeds the watchlist. Idempotent (INSERT OR REPLACE).
- `ingest(records, db_path)` — takes the scraper's list of dicts, generates UUIDs for missing `signal_id`, inserts into `signals`, and skips duplicates that violate the `source_url` unique index. Returns the inserted count.

A private `wipe_signals()` is used by the KPI harness between runs.

agents/

agents/prompts.py

Three constants:

- `SYSTEM_PROMPT` — defines the Signal Analyst's role, the five sectors, six signal categories, three review cycles, and the JSON output schema. Includes explicit alias-resolution guidance ("EMPNG→ExxonMobil", "OTML→Ok Tedi").
- `CLASSIFY_USER_PROMPT_TEMPLATE` — an f-string that injects the comma-separated watchlist canonical names plus the raw posting text.

- `BLOCKLIST_KEYWORDS` — ~30 substrings that pre-filter obvious non-Easy-Skill roles before any LLM call (marketing manager, hospitality, retail, accounting, childcare, etc.). Filtered counts are logged so they can be quoted in §5.2.

agents/signal_analyst.py

The heart of the PoC. Public function `classify_pending(db_path, batch_size=20, gemini_caller=None)`. What it does, in order:

1. Opens SQLite, loads the watchlist (with aliases) into memory.
2. Selects up to `batch_size` rows where `classified_at` IS NULL, oldest first.
3. For each row, runs the pre-filter: drops if `raw_content` < 50 chars or matches the blocklist. Filtered rows still get a `classified_at` stamp (so they don't reappear) and `analysis_notes` records the reason.
4. For surviving rows, builds the user prompt and calls Gemini via `_call_with_retry`. The caller is injected (`gemini_caller=...`) so tests can swap in a fake. Real callers use the `google-genai` SDK with `response_mime_type='application/json'` + a strict JSON schema, so the model is forced to return parseable structured output. Retries: exponential backoff starting at 8 s, up to 5 attempts on 429 `RESOURCE_EXHAUSTED`.
5. Validates and coerces the JSON via `_coerce_classification` — if the model returns an off-schema enum, falls back to safe defaults rather than crashing.
6. Calls `fuzzy_match_watchlist` (`rapidfuzz.WRatio`, threshold 85) on the LLM's `watchlist_match` string to map it onto a canonical watchlist company. Resolves the tier from the canonical entry. If no match, leaves `watchlist_tier` NULL and respects the LLM's `is_new_prospect` flag.
7. Updates the row with the resolved company, sector, category, cycle, tier, prospect flag, reasoning note, and `classified_at`. Sleeps `INTER_CALL_DELAY_SECONDS` (7 s) between successful calls to stay under the rate limit.
8. Returns a Counter dict with totals: `total_pending`, `classified`, per-category counts, `filtered_too_short`, `filtered_blocklist`, `errors`.

delivery/

delivery/digest.py

Builds the Slack-flavoured weekly digest. Public function `build_digest(db_path, since)` queries all rows where `classified_at` is set and `captured_at` >= `since`. Composes five sections, in this exact order, matching the MIOS Sample Reports format:

1. Header — `":large_blue_circle: MIOS Weekly Intelligence — Week of <date>"`
2. Key Signals This Week — up to 10 items grouped by inferred geography (`infer_geography()` runs a keyword sweep over the raw text for PNG markers like Lihir, Porgera, Tabubil). Within each geography, items are ranked: leadership > project > financial > competitive > hiring_velocity > market_intel.
3. Market Pulse — auto-generated bullets summarising the dataset: total signals, top sector, AU/PNG split, review-cycle mix, count of new prospects.
4. Hiring Velocity — Top 10 Watchlist Clients — a code-block table of watchlist companies by signal count this week.
5. New Names (Not in Watchlist) — a table of distinct companies where `is_new_prospect=1`.

delivery/slack.py

Single function `post_digest(webhook_url, digest_markdown)`. POSTs `{"text": ..., "mrkdwn": true}` to the incoming webhook with a 15 s timeout. Returns `True` on HTTP 200, `False` otherwise. Never raises — the brief requires the pipeline to keep running if Slack is down.

scraper/

scraper/pngworkforce.py

Two layers, separated for testability:

- `parse_listing(html, source_url, base_url)` — pure parser. Uses BeautifulSoup to look for common job-card patterns (`article.job-listing`, `div.job-card`, etc.). Falls back to any `<article>` with a heading + link. Extracts title, location, body, resolves relative URLs against the base. Tested against a saved HTML fixture (`tests/fixtures/pngworkforce_listing.html`) so it runs offline.
- `scrape(limit=200, base_url=None)` — live fetcher. Uses crawlee's BeautifulSoupCrawler with a polite User-Agent. On any exception — site change, timeout, malformed HTML — logs a warning and returns `[]`. Per the brief, the pipeline must continue on synthetic data when scraping fails.

evaluation/

evaluation/kpi_harness.py

The orchestrator. Public function `run_evaluation(db_path, ground_truth_path, runs=5)`. What happens, per run:

1. Wipes the `signals` table (watchlist preserved).
2. Loads all 80 records from `data/synthetic_postings.jsonl`, adapts them to the ingest record shape, and bulk-inserts.
3. Starts a `perf_counter`, runs `classify_pending`, stops the clock.
4. Calls `score_run()`: counts a row as correct only if *both* `signal_category` and `review_cycle` match the ground truth. Watchlist precision = of rows predicted as a watchlist match, fraction where the predicted company equals the ground-truth company.
5. Records: classification accuracy %, watchlist precision %, latency seconds, API call count, AU\$0.00 (free tier), filter counts, error count.

After all runs, `_aggregate()` computes the means and stdev. `_write_csv()` writes per-run rows + an aggregate row to `results.csv`. `_print_kpi_table()` prints a markdown table with Target / Achieved / Status columns matching the §5.2 layout. The CLI `main()` then builds the digest and posts it to Slack unless `--no-slack` is passed.

data/

data/synthetic_postings.jsonl

80 hand-authored job postings used as the labelled ground-truth set. Distribution:

- 48 (60%) clearly relevant, watchlist company, clean classification — spread across BHP, Rio Tinto, Newmont, Barrick, TotalEnergies, ExxonMobil, Glencore, Downer, Monadelphous, Ok Tedi, Vale, Eramet, Saipem, Technip, Subsea7, Aurecon, Vinci, Thales, Safran, Eiffage.

- 20 (25%) clearly irrelevant — hotel concierge, marketing manager, retail merchandiser, etc. Should classify as sector other and not match the watchlist.
- 12 (15%) edge cases — ambiguous "General Manager — Operations" titles at watchlist and non-watchlist companies, alias variants (Newcrest Lihir, TechnipFMC, EMPNG, OTML), 2 bilingual EN/Tok Pisin postings, and 4 new-prospect companies (Kumul Petroleum, Hidden Valley Mine, Coronado Global Resources, Pilbara Minerals).

Each line is JSON with keys: `id`, `source_url`, `raw_text`, and the six `ground_truth_*` labels.

tests/

tests/test_loader.py (6 tests)

Schema creation, watchlist seeding, idempotency, dedupe by `source_url`, empty-content rejection, `wipe_signals` behaviour.

tests/test_signal_analyst.py (15 tests)

Pre-filter (too short, blocklist, pass-through), fuzzy watchlist matching (exact, alias, typo, unknown, None input), enum-coercion fallback, end-to-end classification with a fake Gemini caller (happy path, prefilter, alias resolution, new prospect, error handling).

tests/test_digest.py (10 tests)

Geography inference, all five sections present, geography grouping order, exclusion of unclassified rows, since-window respect, new-prospect table, plus Slack post (success, non-200, empty URL, `RequestException`). Slack uses `unittest.mock.patch` on `requests.post`.

tests/test_scraper.py (8 tests)

Drives `parse_listing` against `tests/fixtures/pngworkforce_listing.html` — no live network. Verifies card detection, URL resolution, title/location extraction, empty-input handling, and that `scrape()` returns `[]` on invalid URL or crawler exceptions.

3. What happens when you run `kpi_harness`

End-to-end trace of `python -m evaluation.kpi_harness --runs 1`:

Step 1 — Argparse + logging.

`main()` parses CLI flags, `configure_logging()` sets the root logger to `LOG_LEVEL`.

Step 2 — Load ground truth.

`load_ground_truth()` reads `data/synthetic_postings.jsonl`, returns 80 dicts.

Step 3 — `init_db`.

`loader.ingest.init_db` runs `schema.sql` against `data/mios.db` (creates tables, indexes), then seeds the watchlist (20 rows). Idempotent — safe across runs.

Step 4 — Wipe signals.

`wipe_signals()` truncates the signals table. Watchlist is preserved.

Step 5 — Ingest ground truth.

`ingest()` writes all 80 records into `signals` with auto UUIDs and dedupe-by-URL. `classified_at` remains NULL on every row.

Step 6 — Classification clock starts.

`perf_counter()` snapshot for the latency KPI.

Step 7 — `classify_pending`.

Loops over the 80 pending signals: pre-filter drops 20 blocklisted rows; the remaining 60 are sent to Gemini one at a time with a 7 s sleep between calls. Each response is JSON-parsed, schema-coerced, and the company name is fuzzy-matched against the watchlist. The signals row is updated with `classification + tier + reasoning + classified_at`.

Step 8 — Latency captured.

`perf_counter()` stop. Latency is recorded for this run.

Step 9 — Score the run.

`score_run()` joins classified rows back to ground truth on `signal_id`, computes classification accuracy and watchlist precision.

Step 10 — `results.csv`.

`_write_csv()` writes the per-run row + an aggregate row (means + stdev) to `results.csv` at the repo root.

Step 11 — KPI table to stdout.

`_print_kpi_table()` emits the §5.2 markdown table with Target / Achieved / Status.

Step 12 — Build the digest.

`delivery.digest.build_digest()` queries the classified rows captured in the last 7 days and renders the five Slack-mrkdown sections.

Step 13 — Post to Slack.

`delivery.slack.post_digest()` POSTs the digest to `SLACK_WEBHOOK_URL`. Logs success/failure but never raises.

Step 14 — Exit.

`main()` returns 0 if any rows were classified, 1 otherwise.

Note. Quota tip. Gemini 2.5 Flash free tier is approximately 200 requests per day. One full run uses about 60 calls. Five runs in one day will exhaust the daily allowance — either spread runs across days or set `GEMINI_MODEL=gemini-2.5-flash-lite` to use the separate quota pool.